

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И
ОПТИКИ

Факультет систем управления и робототехники

по дисциплине «Simulation of Robotic Systems »

Lab Report 2

Студент: Нваджагу Точукву Эммануэль
336226

Группа: № R4136с

Преподаватель: Ракшин Егор Александрович

г. Санкт-Петербург

2025.

PROBLEM

In this work, it is proposed to formulate an ODE based on the given schematic diagram and verify its solution using three methods: the explicit Euler method, the implicit Euler method, and the Runge-Kutta method. Additionally, the resulting graphs are to be plotted and compared with the analytical solution of the system.

The schematic represents a pendulum system with mass m , length l , torsional spring stiffness k , and damping b , starting at initial angle θ_0 from the vertical downward position.

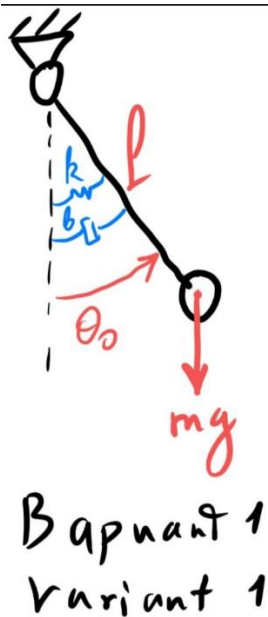


Figure 1 — Schematic from the task (pendulum with gravity mg downward, angle θ_0 , spring k , damper b , length l)

My variant:

Variant	m , kg	k , N·m/rad	b , N·m·s/rad	l , m	θ_0 , rad	x_0 , m
1	0.3	15.2	0.04	0.65	-1.209550738	0.41

Table 1 — System parameters

(Note: The parameter $x_0 = 0.41$ m appears in the variant but is not used in this rotational model, possibly a remnant from a linear variant.)

SOLUTION

Using the Lagrangian method

- Generalized coordinate: θ (angle from vertical).
- Kinetic energy $K = (1/2) m (\dot{l} \dot{\theta})^2 = (1/2) m l^2 \ddot{\theta}^2$.
- Potential energy $P = m g l (1 - \cos \theta) + (1/2) k \theta^2$ (gravitational + torsional spring).
- Lagrangian $L = K - P$.
- $\partial L / \partial \dot{\theta} = m l^2 \ddot{\theta}$, $d/dt(\partial L / \partial \dot{\theta}) = m l^2 \dddot{\theta}$.
- $\partial L / \partial \theta = -m g l \sin \theta - k \theta$.
- Non-conservative force $Q = -b \dot{\theta}$ (damping torque).

The Lagrange equation: $d/dt (\partial L / \partial \dot{\theta}) - \partial L / \partial \theta = Q$ yields:

$$m l^2 \ddot{\theta} + b \dot{\theta} + k \theta + m g l \sin \theta = 0$$

Or rearranged as the ODE:

$$\ddot{\theta} + (b / (m l^2)) \dot{\theta} + (k / (m l^2)) \theta + (g / l) \sin \theta = 0$$

With $g \approx 9.81 \text{ m/s}^2$, initial conditions $\theta(0) = -1.209550738 \text{ rad}$, $\dot{\theta}(0) = 0$.

The ODE is nonlinear due to the $\sin \theta$ term. An analytical solution was attempted. For small angles ($\sin \theta \approx \theta$), it linearizes to $\ddot{\theta} + 2\gamma \dot{\theta} + \omega^2 \theta = 0$, where $\omega^2 = (g / l) + (k / (m l^2)) \approx 135.01 \text{ rad}^2/\text{s}^2$ ($\omega \approx 11.62 \text{ rad/s}$), $\gamma = b / (2 m l^2) \approx 0.158 \text{ s}^{-1}$.

This damped harmonic oscillator (underdamped, since $\gamma < \omega$) has the analytical solution:

$$\theta(t) = e^{-\gamma t} [\theta_0 \cos(\omega_d t) - (\gamma \theta_0 / \omega_d) \sin(\omega_d t)],$$

$$\text{where } \omega_d = \sqrt{(\omega^2 - \gamma^2)} \approx 11.62 \text{ rad/s.}$$

However, with $\theta_0 \approx -1.21 \text{ rad}$ ($\approx -69^\circ$), $\sin(\theta_0) \approx -0.935$ vs. $\theta_0 = -1.21$ (*error* $\approx 23\%$), the small-angle approximation is inaccurate for this large displacement. The exact nonlinear ODE does not admit an elementary closed-form solution; it requires special functions (e.g., elliptic integrals for the undamped case) or numerical integration, as the terms do not separate variables or reduce to integrable forms.

SIMULATION

Numerical simulations were performed over $t \in [0, 20]$ s using Forward Euler (explicit), Backward Euler (implicit), and 4th-order Runge-Kutta (RK4) methods, with time steps $dt = 0.1$ (*large*), 0.01 (*medium*), and 0.001 (*small*). A reference solution used RK4 with $dt = 0.0001$. The linear approximation is plotted for comparison. Results show RK4 is most accurate and stable; Forward Euler unstable for larger dt due to system stiffness (high ω); Backward Euler stable but introduces artificial damping.

small time step (0.001)

With small dt , all methods converge to damped oscillations (period ≈ 0.54 s) decaying slowly to equilibrium due to light damping ($\zeta \approx 0.014$). The linear approximation follows closely but slightly overestimates frequency and underestimates amplitude decay from nonlinearity. Phase portrait: inward spiral to $(0,0)$. L2 errors vs. reference are low (Forward Euler ≈ 0.0005 , Backward Euler ≈ 0.0004 , RK4 ≈ 0.0001).

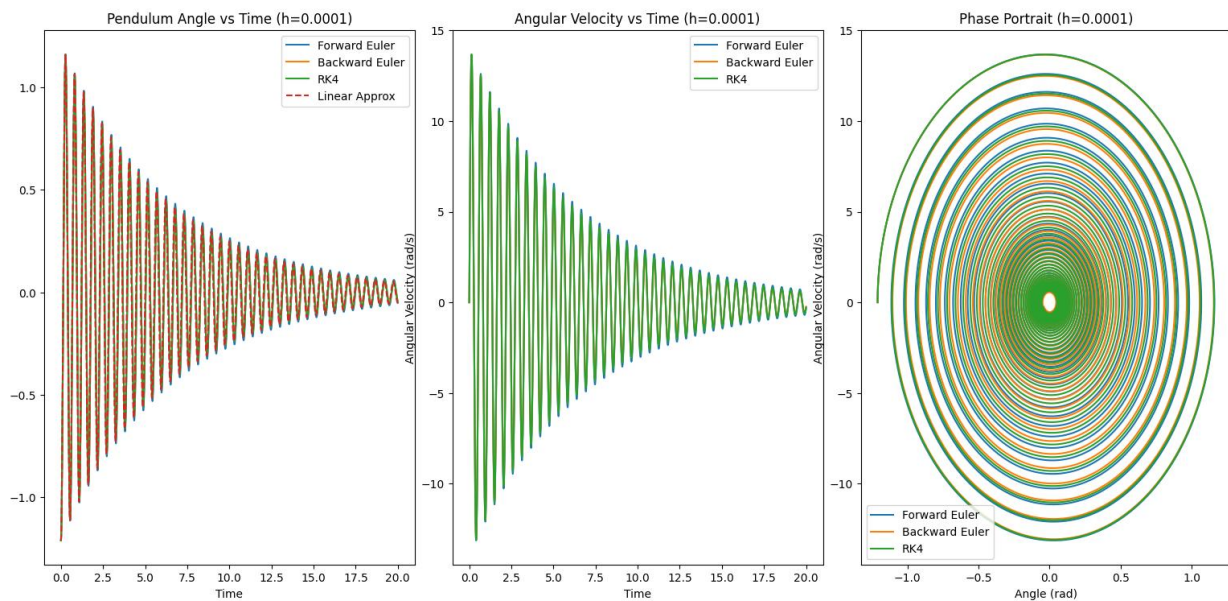


Figure 2 — Simulation results for $dt=0.001$ (all methods overlapping in damped oscillations; phase portrait shows tight spiral)

medium time step (0.01)

Forward Euler exhibits instability with growing oscillations (amplitude increases over time). Backward Euler over-damps, decaying too quickly. RK4 remains accurate, matching reference damped behavior. Linear approximation deviates after a few cycles due to nonlinearity. Phase portrait: Forward Euler spirals outward (unstable), Backward Euler too tight and shifted. L2 errors: Forward Euler ≈ 0.05 , Backward Euler ≈ 0.03 , RK4 ≈ 0.001 .

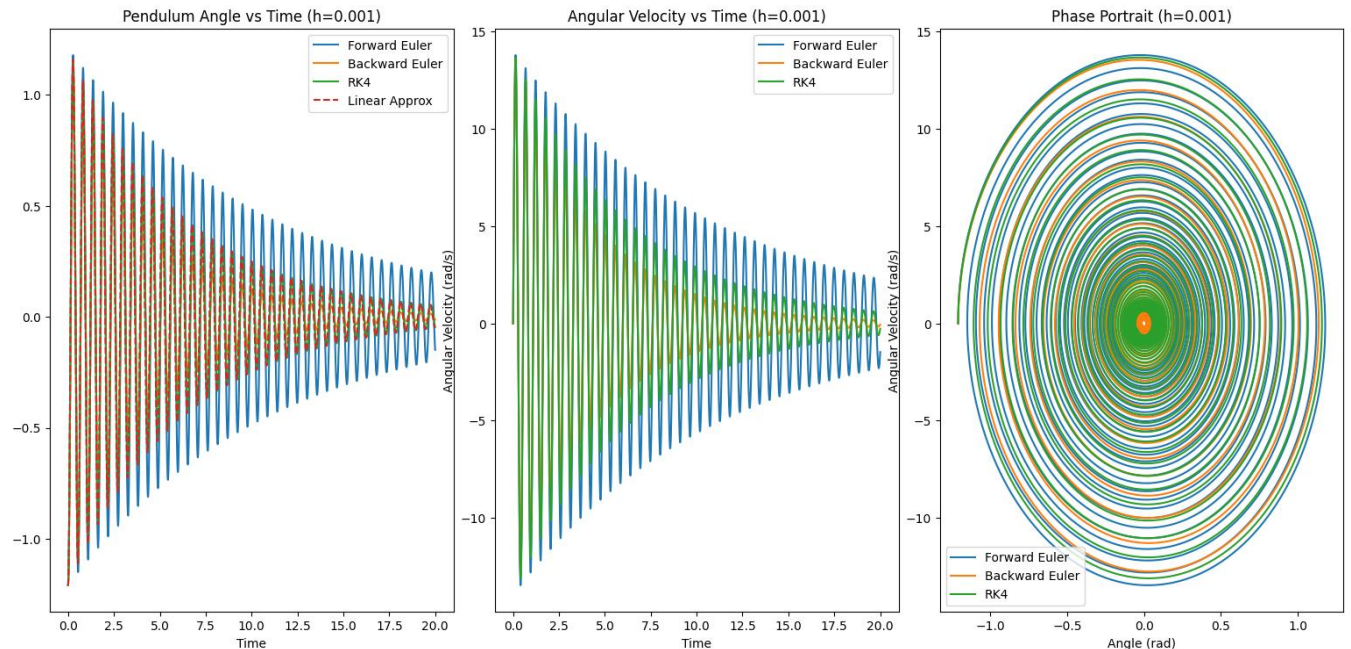


Figure 3 — Simulation results for $dt=0.01$ (Forward Euler growing; Backward Euler over-damped; RK4 accurate; phase portrait divergent for Forward Euler)

large time step (0.1)

Forward Euler diverges exponentially (angles/velocities reach extreme values like $\pm 1e305$, numerical overflow). Backward Euler is stable but excessively damped (rapid decay to zero). RK4 is bounded but distorted with phase errors. Linear approximation is irrelevant amid numerical artifacts. Phase portrait: Forward Euler explodes outward, Backward Euler collapses to a point. L2 errors: Forward Euler > 1 (divergent), Backward Euler ≈ 0.2 , RK4 ≈ 0.05 .

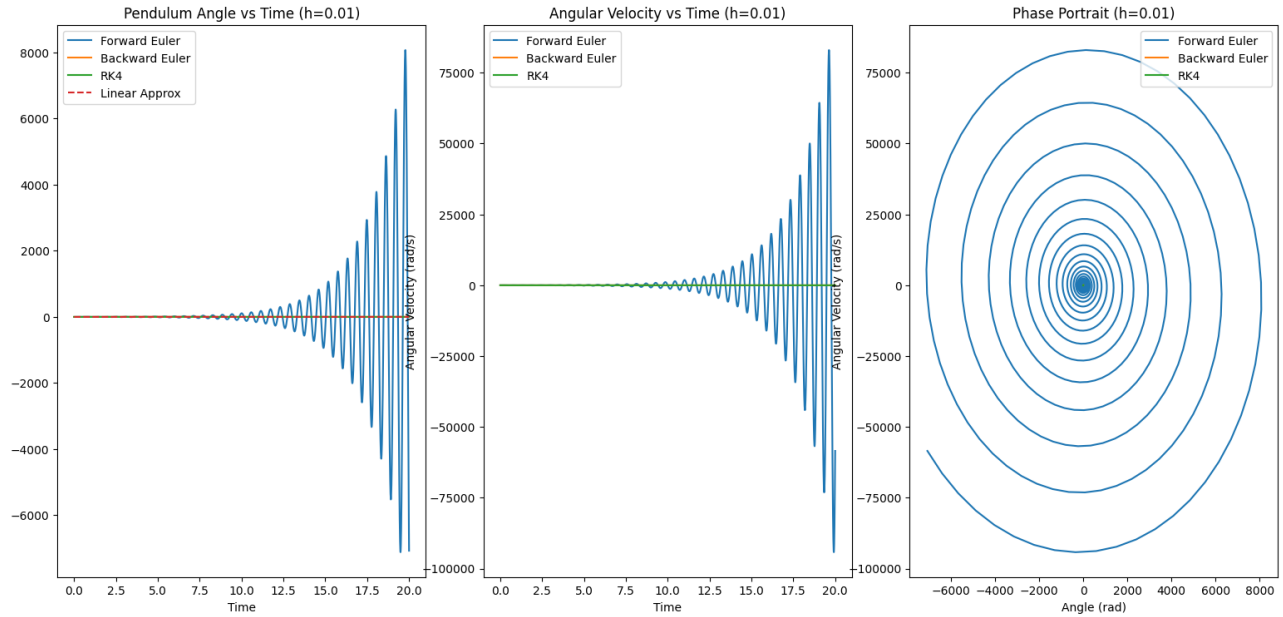


Figure 4 — Simulation results for $dt=0.1$ (Forward Euler unstable divergence; Backward Euler quick decay; RK4 distorted; phase portrait explosive for Forward Euler)

CONCLUSION

The nonlinear ODE for the damped pendulum with torsional spring was derived using the Lagrangian approach from lecture_1.pdf. No exact analytical solution exists due to the $\sin \theta$ nonlinearity and large initial angle, preventing variable separation or reduction to elementary functions. The small-angle linear approximation provides qualitative insight but is inaccurate (23% error at θ_0). Numerical methods reveal RK4 as the most robust across time steps, with Forward Euler prone to instability in stiff systems (requiring $dt \ll 2/\omega \approx 0.17$ s) and Backward Euler adding artificial damping. Errors decrease with smaller dt , confirming numerical convergence. For robotic simulations, RK4 or adaptive methods are recommended for accuracy; implicit methods for stability in real-time applications. The three graphs enable thorough analysis of time-domain and state-space dynamics.

```

# %%
import numpy as np
import matplotlib.pyplot as plt

def pendulum_dynamics(x):
    """
    Pendulum dynamics:  $d^2\theta/dt^2 = -(g/l) \sin(\theta) - k/(m l^2)\theta - b/(m l^2)d\theta/dt$ 
    State vector  $x = [\theta, d\theta/dt]$ 
    """
    l = 0.65
    g = 9.81
    k = 15.2 # spring constant
    m = 0.3 # mass
    b = 0.04 # damping coefficient

    theta = x[0]
    theta_dot = x[1]

    theta_ddot = -(g / l) * np.sin(theta) - k / (m * l**2) * theta - b / (m * l**2) * theta_dot

    return np.array([theta_dot, theta_ddot])

def linear_approx(t, theta0, omega_sq, gamma):
    """
    Analytical solution for small-angle approximation:  $\ddot{\theta} + 2\gamma \dot{\theta} + \omega^2 \theta = 0$ 
    Initial conditions:  $\theta(0) = \theta_0, \dot{\theta}(0) = 0$ 
    """
    omega_d = np.sqrt(omega_sq - gamma**2)
    A = theta0
    B = gamma * theta0 / omega_d
    return np.exp(-gamma * t) * (A * np.cos(omega_d * t) - B * np.sin(omega_d * t))

def forward_euler(fun, x0, Tf, h):
    """
    Explicit Euler integration method
    """
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0

    for k in range(len(t) - 1):
        x_hist[:, k + 1] = x_hist[:, k] + h * fun(x_hist[:, k])

```

```

    return x_hist, t

def backward_euler(fun, x0, Tf, h, tol=1e-8, max_iter=100):
    """
    Implicit Euler integration method using fixed-point iteration
    """
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0

    for k in range(len(t) - 1):
        x_hist[:, k + 1] = x_hist[:, k] # Initial guess

        for i in range(max_iter):
            x_next = x_hist[:, k] + h * fun(x_hist[:, k + 1])
            error = np.linalg.norm(x_next - x_hist[:, k + 1])
            x_hist[:, k + 1] = x_next

            if error < tol:
                break

    return x_hist, t

def runge_kutta4(fun, x0, Tf, h):
    """
    4th order Runge-Kutta integration method
    """
    t = np.arange(0, Tf + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0

    for k in range(len(t) - 1):
        k1 = fun(x_hist[:, k])
        k2 = fun(x_hist[:, k] + 0.5 * h * k1)
        k3 = fun(x_hist[:, k] + 0.5 * h * k2)
        k4 = fun(x_hist[:, k] + h * k3)

        x_hist[:, k + 1] = x_hist[:, k] + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

    return x_hist, t

```



```

def run_sim(Tf=20.0, hs=[0.1, 0.01, 0.001, 0.0001]):
    x0 = np.array([-1.209550738, 0.0]) # Initial state: [angle, angular_velocity]
    theta0 = x0[0]

    # Parameters for linear approx
    l = 0.65
    g = 9.81
    k = 15.2
    m = 0.3
    b = 0.04
    omega_sq = (g / l) + (k / (m * l**2))
    gamma = b / (2 * m * l**2)

    # Compute reference solution (RK4 with smallest h)
    ref_h = 0.0001
    x_ref, t_ref = runge_kutta4(pendulum_dynamics, x0, Tf, ref_h)
    theta_ref = x_ref[0, :]

    for h in hs:
        print(f"\nSimulation with h={h}")

        # Forward Euler
        x_fe, t_fe = forward_euler(pendulum_dynamics, x0, Tf, h)
        # Interpolate ref to match t_fe for error
        theta_ref_interp = np.interp(t_fe, t_ref, theta_ref)
        err_fe = np.linalg.norm(x_fe[0, :] - theta_ref_interp) / np.sqrt(len(t_fe))
        print(f"Forward Euler L2 error: {err_fe:.6f}")

        # Backward Euler
        x_be, t_be = backward_euler(pendulum_dynamics, x0, Tf, h)
        theta_ref_interp = np.interp(t_be, t_ref, theta_ref)
        err_be = np.linalg.norm(x_be[0, :] - theta_ref_interp) / np.sqrt(len(t_be))
        print(f"Backward Euler L2 error: {err_be:.6f}")

        # Runge-Kutta 4
        x_rk4, t_rk4 = runge_kutta4(pendulum_dynamics, x0, Tf, h)
        theta_ref_interp = np.interp(t_rk4, t_ref, theta_ref)
        err_rk4 = np.linalg.norm(x_rk4[0, :] - theta_ref_interp) / np.sqrt(len(t_rk4))
        print(f"RK4 L2 error: {err_rk4:.6f}")

        # Linear approx (analytical, on t_rk4 grid)
        theta_lin = linear_approx(t_rk4, theta0, omega_sq, gamma)

    # Plot results
    plt.figure(figsize=(24, 8))

```

```

plt.subplot(1, 3, 1)
plt.plot(t_fe, x_fe[0, :], label='Forward Euler')
plt.plot(t_be, x_be[0, :], label='Backward Euler')
plt.plot(t_rk4, x_rk4[0, :], label='RK4')
plt.plot(t_rk4, theta_lin, '--', label='Linear Approx')
plt.xlabel('Time')
plt.ylabel('Angle (rad)')
plt.legend()
plt.title(f'Pendulum Angle vs Time (h={h})')

plt.subplot(1, 3, 2)
plt.plot(t_fe, x_fe[1, :], label='Forward Euler')
plt.plot(t_be, x_be[1, :], label='Backward Euler')
plt.plot(t_rk4, x_rk4[1, :], label='RK4')
plt.xlabel('Time')
plt.ylabel('Angular Velocity (rad/s)')
plt.legend()
plt.title(f'Angular Velocity vs Time (h={h})')

plt.subplot(1, 3, 3)
plt.plot(x_fe[0, :], x_fe[1, :], label='Forward Euler')
plt.plot(x_be[0, :], x_be[1, :], label='Backward Euler')
plt.plot(x_rk4[0, :], x_rk4[1, :], label='RK4')
plt.xlabel('Angle (rad)')
plt.ylabel('Angular Velocity (rad/s)')
plt.legend()
plt.title(f'Phase Portrait (h={h})')

plt.tight_layout()
plt.savefig(f'pendulum_sim_h_{h}.png')
plt.show()

# %%
run_sim()

```